Universidad Europea



Procesamiento y Escalabilidad

Arquitectura Big Data

Fundamentos de Big Data

Grado en Física

Universidad Europea

“¿Qué preferiría una empresa de streaming como Netflix: tener los datos procesados al instante aunque no sean exactos, o procesarlos con más precisión aunque tarden minutos?”

Inmediatez (streaming) vs. Precisión (batch).

Universidad Europea

Conclusión

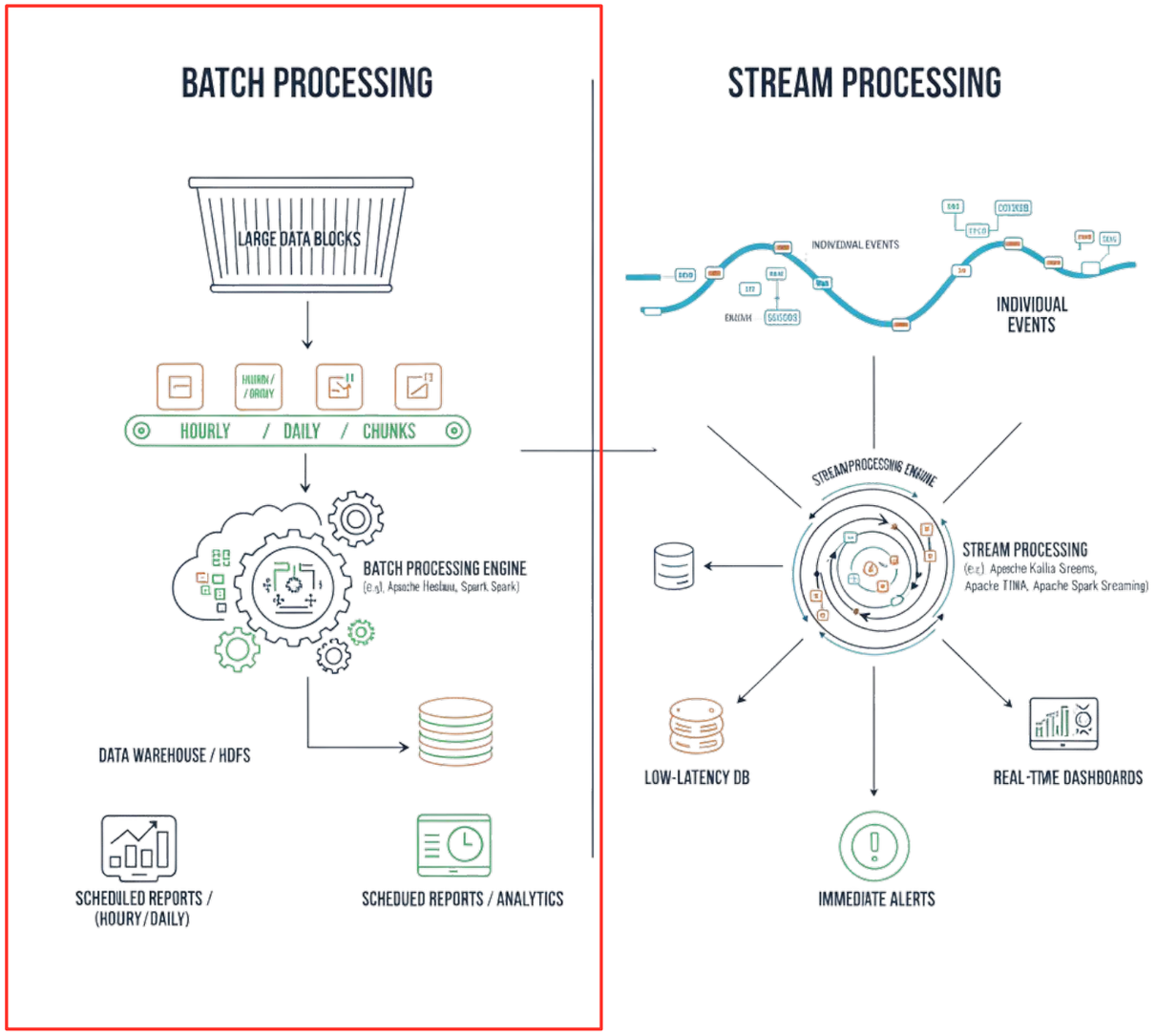
Ambas son necesarias. Las Arquitecturas híbridas (Lambda, Kappa) permiten combinarlas.

CONTENIDOS

1. **Procesamiento de Datos.**
2. Escalabilidad en Big Data.
3. Tolerancia a fallos y alta disponibilidad.
4. Casos de Uso.

1.1. Batch Processing

Método tradicional para gestionar grandes volúmenes de datos. En él, los "Large Data Blocks" son recolectados y procesados en grupos o "batches" en intervalos regulares (por ejemplo, cada hora, día, o en fragmentos).



1.1. Batch Processing

- Recolección de datos: los datos se acumulan durante un período de tiempo. No se procesan en el momento en que se generan, sino que esperan hasta que se haya recogido una cantidad predeterminada o se haya alcanzado un cierto tiempo..
- Batch Processing Engine: una vez que el batch está listo, se envía a un motor de procesamiento como Apache Hadoop o Apache Spark. Estos motores están optimizados para procesar grandes conjuntos de datos de manera eficiente.

1.1. Batch Processing

- Almacenamiento: los datos procesados a menudo se almacenan en un "Data Warehouse / HDFS" (Hadoop Distributed File System), que es adecuado para almacenar grandes volúmenes de datos estructurados y semiestructurados.
- Salida y uso: los resultados de este procesamiento se utilizan típicamente para realizar informes y análisis programados, que pueden realizarse por hora, día, semana, etc.

1.1. Batch Processing

Características

- Procesa bloques históricos completos.
- **Alta precisión**, pero **alta latencia**.
- Ideal para reportes, analítica descriptiva e histórica.

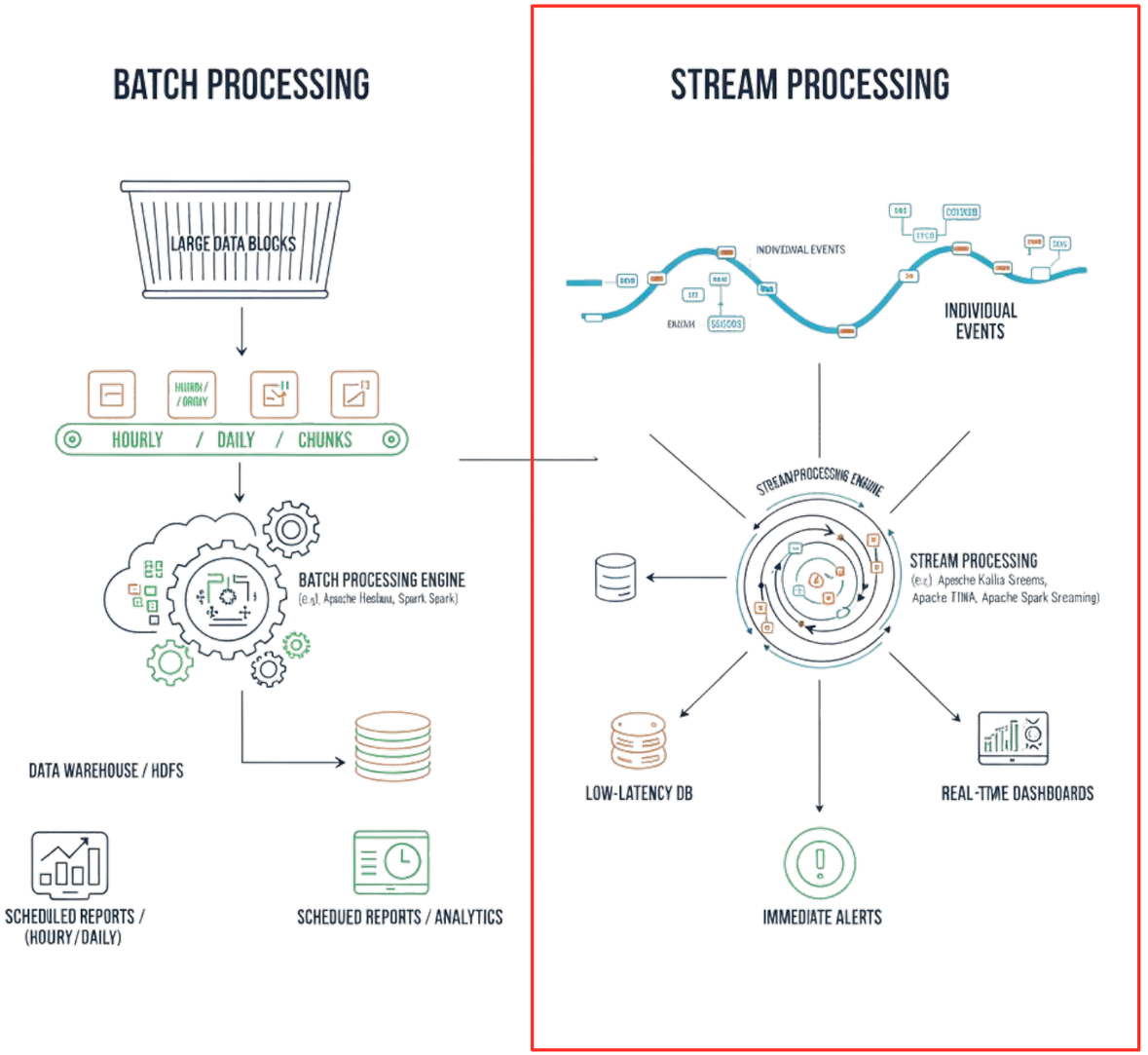
Herramientas

- Hadoop MapReduce..
- Apache Spark (batch).

***Ejemplo**: cálculo de reportes financieros diarios o mensuales.*

1.2. Streaming Processing

A diferencia del Batch Processing, el procesamiento en streaming maneja "Individual Events" a medida que llegan. El enfoque aquí es el tiempo real o casi real.



1.2. Streaming Processing

- Flujo continuo de datos: Los datos fluyen continuamente en forma de pequeños paquetes o eventos individuales. Cada evento se procesa a medida que se recibe, sin esperar a que se acumulen grandes bloques.
- Streaming Processing Engine: Un motor de procesamiento en streaming (como Apache Kafka Streams o Apache Flink) ingiere estos eventos individuales y los procesa al instante.

1.2. Streaming Processing

- Almacenamiento de baja latencia (Low-Latency DB): los resultados intermedios o finales pueden almacenarse en una base de datos de baja latencia para un acceso rápido.
- Salida y uso en tiempo real: los resultados del procesamiento en streaming se utilizan para "Real-Time Dashboards" y/o "Immediate Alerts". Esto permite tomar decisiones y acciones rápidas basadas en los datos más recientes.

1.2. Streaming Processing

Características

- **Baja latencia**, decisiones instantáneas.
- Procesa flujos continuos (eventos).
- **Precisión menor**, pero alta **inmediatez**.

Herramientas

- Apache Kafka.
- Apache Flink / Spark Streaming.

***Ejemplo**: Recomendaciones en vivo (Netflix, YouTube).*

1.3. Batch vs. Streaming

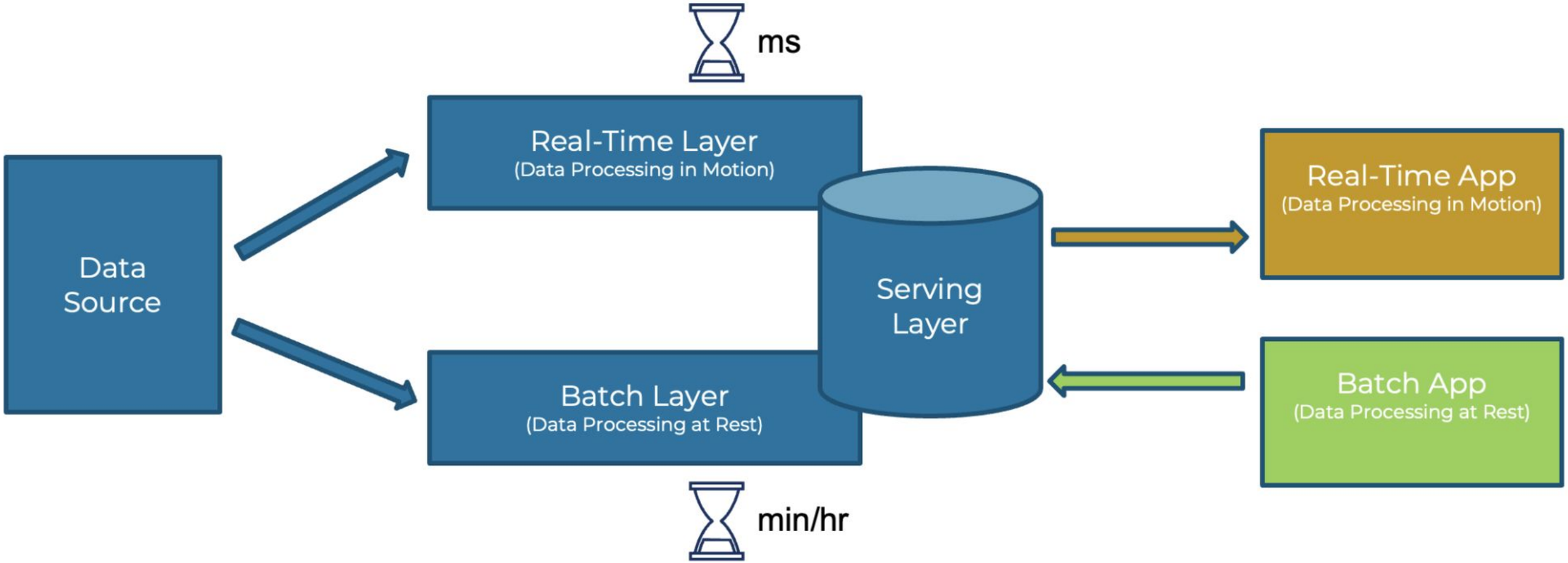
Criterio	Batch	Streaming
Latencia	Alta (minutos u horas)	Baja (milisegundos o segundos)
Volumen	Masivo, histórico	Flujo constante
Precisión	Alta	Menor (aproximaciones o muestreo)
Costo	Más económico	Más costoso (infraestructura 24/7)
Casos de uso	Análisis histórico, informes	Detección de eventos, monitorización
Ejemplo	Informes de ventas mensuales	Detección de fraude en tiempo real

Importante: as Arquitecturas híbridas (Lambda, Kappa) combinan Batch y Streaming.
+info: Kappa Architecture is Mainstream Replacing Lambda.

Universidad Europea

Lambda Architecture

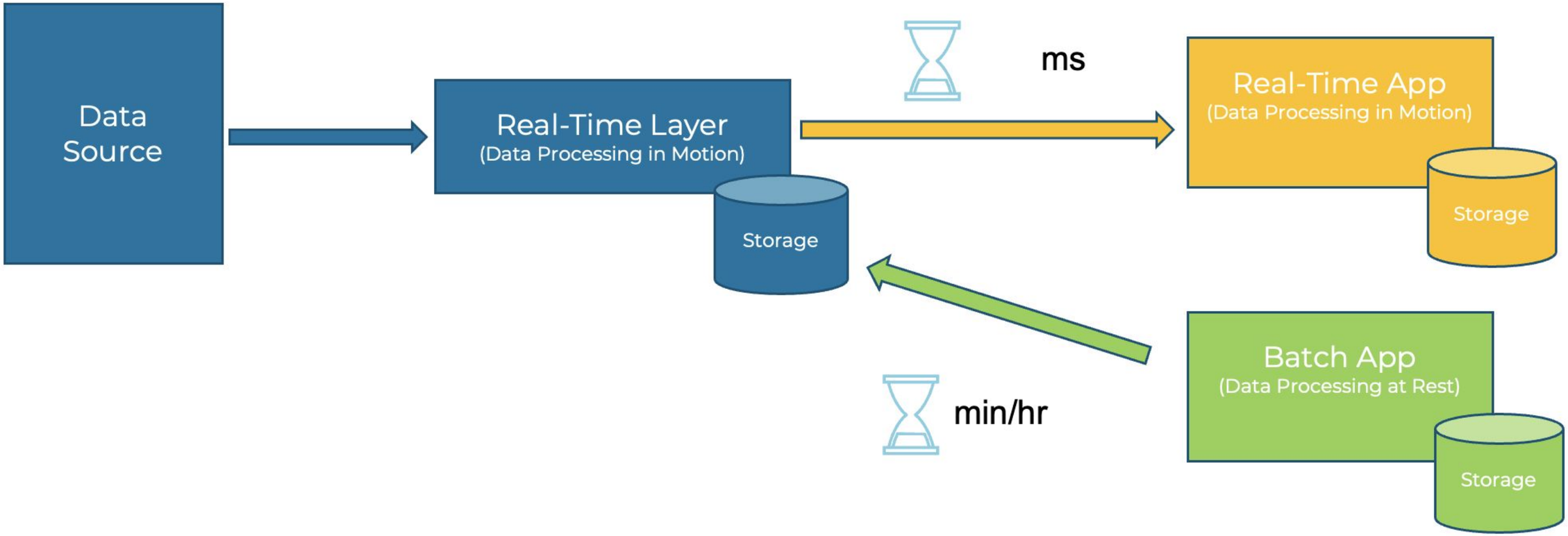
Option 1: Unified serving layer



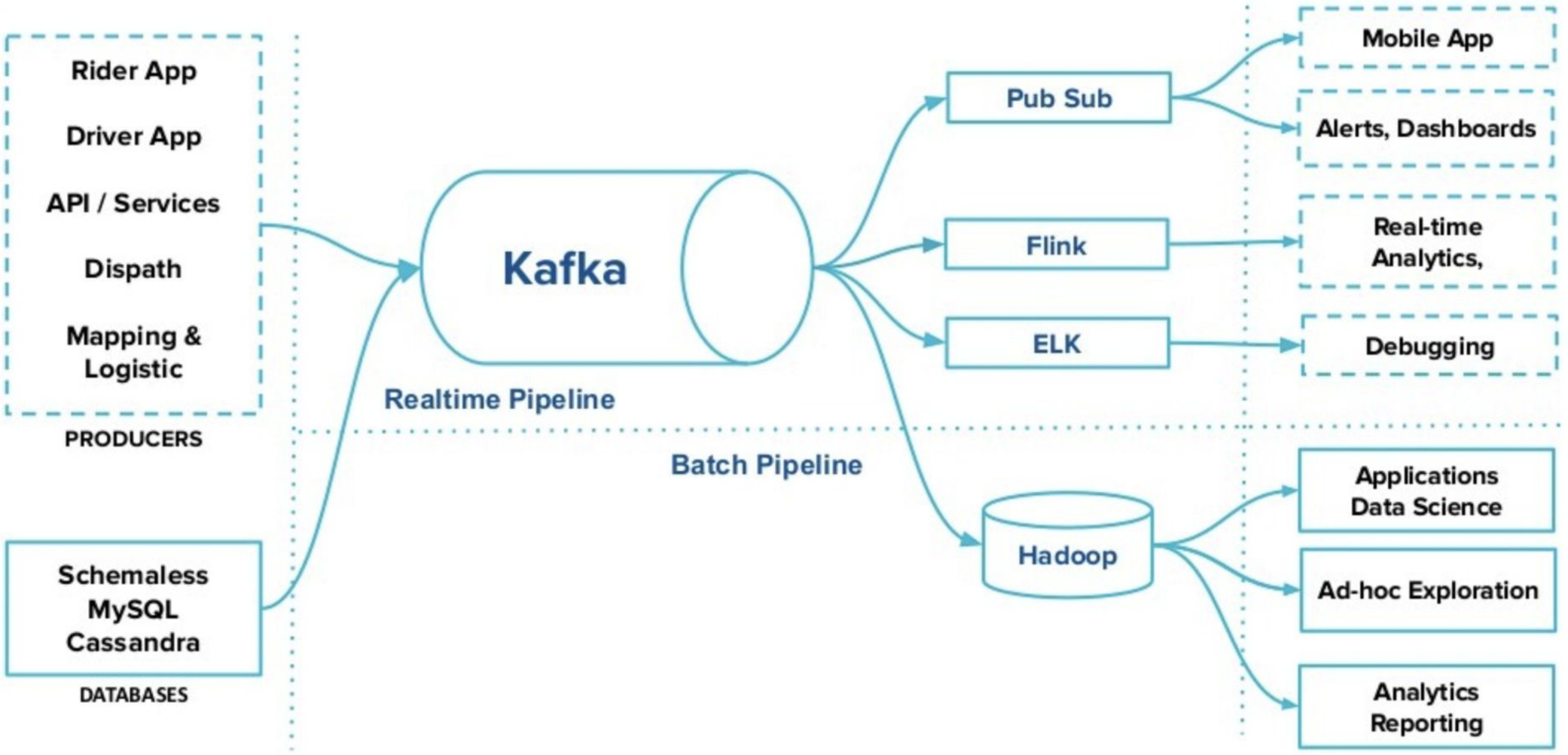
Universidad Europea

Kappa Architecture

One pipeline for real-time and batch consumers



Kafka at Uber



CONTENIDOS

1. Procesamiento de Datos.
2. **Escalabilidad en Big Data.**
3. Tolerancia a fallos y alta disponibilidad.
4. Casos de Uso.

Universidad Europea

2. Escalabilidad en Big Data

- *Capacidad de un sistema para manejar de manera eficiente el creciente* ***volumen, velocidad y variedad*** (¿de que nos suena esto?) *de datos que caracterizan a las aplicaciones modernas.*
- *Capacidad del sistema para* ***crecer en rendimiento sin perder eficiencia*** *ante un aumento de datos o usuarios.*
- *Es uno de los* ***pilares del Big Data*** *junto a la tolerancia a fallos y el procesamiento distribuido.*

2. Escalabilidad en Big Data

Impacto

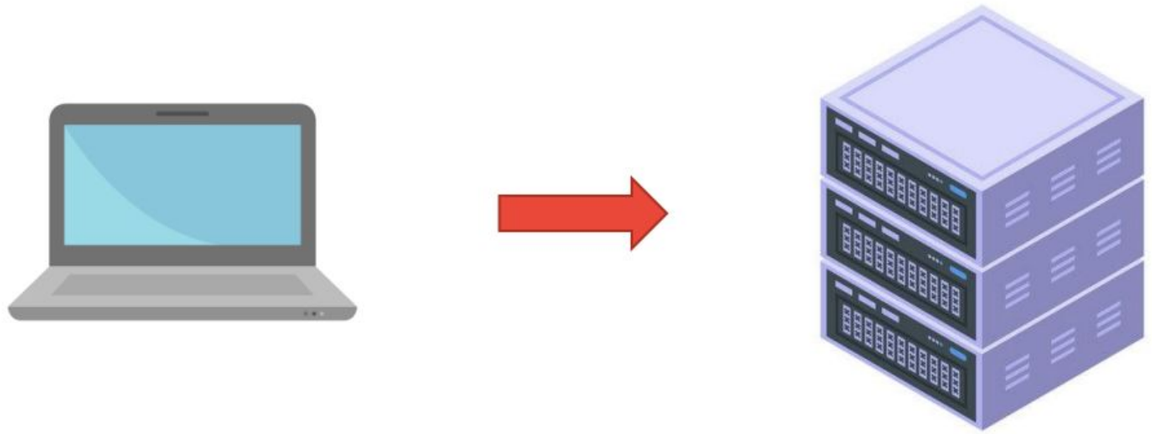
- ***Rendimiento***.
- ***Eficiencia operativa***.

*Un sistema escalable permite **manejar incrementos en la demanda** sin una gran pérdida de rendimiento, así como **optimizar costos**, invirtiendo gradualmente en infraestructura adicional conforme crecen sus necesidades.*

2.1. Escalabilidad Vertical

Vertical (Scale-up)

Añadir más potencia a una máquina existente (por ej., más RAM, CPU), sin cambiar la infraestructura del sistema.



2.1. Escalabilidad Vertical

Ventajas

- Fácil de implementar.
- No requiere modificar la arquitectura.

Desventajas

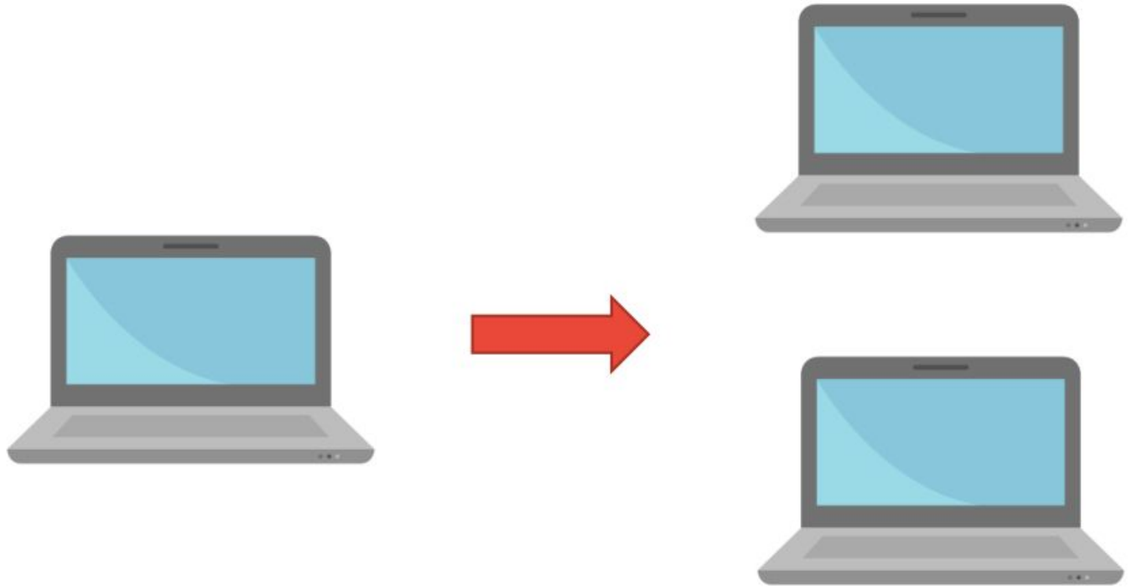
- Limitada por el hardware.
- Costosa a gran escala.

***Ejemplo**: añadir memoria a un servidor de base de datos.*

2.2. Escalabilidad Horizontal

Horizontal (Scale-out)

Añadir más servidores o equipos al sistema existente. Ejemplo: Clústeres de servidores.



Clúster: conjunto de ordenadores separados físicamente y conectadas entre sí por una red de comunicaciones de alta velocidad; cada máquina posee sus componentes de hardware y software que el programador percibe como un solo sistema.

2.2. Escalabilidad Horizontal

Ventajas

- Escalado casi infinito.
- Alta disponibilidad.

Desventajas

- Complejidad de configuración.
- Mayor coste inicial.

***Ejemplo**: añadir 10 nodos al clúster de Spark para procesar datos paralelamente.*

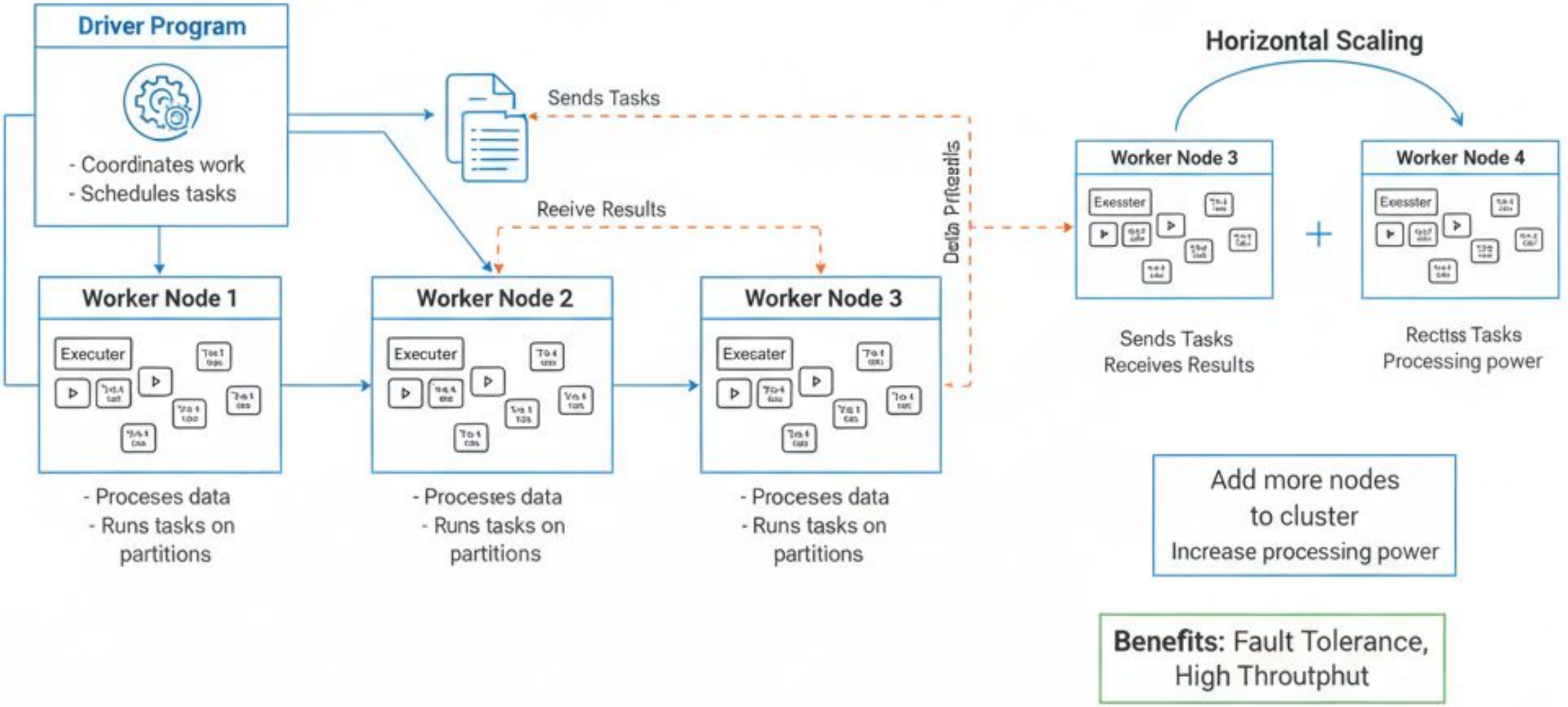
2.3. Escalabilidad en entornos Cloud

Capacidad de los sistemas cloud (AWS, Azure, GCP) para ***ajustar recursos automáticamente***.

Permite crecer o reducir nodos según la demanda.

Ejemplo: *durante el Black Friday, AWS autoscale lanza más instancias EC2 para absorber picos de tráfico.*

Escalabilidad de la Arquitectura Spark



2.4. Garantías de la Escalabilidad

- *Disponibilidad* : acceso constante y operatividad del servicio.
- *Fiabilidad*: consistencia y predictibilidad en el rendimiento del servicio.
- *Capacidad*: habilidad del servicio para manejar la carga del trabajo.
- *Seguridad*: protección contra amenazas y accesos no autorizados.

CONTENIDOS

1. Procesamiento de Datos.
2. Escalabilidad en Big Data.
3. **Tolerancia a fallos y alta disponibilidad.**
4. Casos de Uso.

Universidad Europea

3. Tolerancia a fallos y alta Disponibilidad

- Capacidad de un sistema para **continuar operando correctamente en caso de fallo** de uno o más de sus componentes.
- Es crucial en la infraestructura Big Data debido a la necesidad de mantener la **disponibilidad** y la **integridad** de los datos en todo momento.
- Para lograrlo, los sistemas deben ser diseñados con **redundancia**, capacidad de **replicación** y **mecanismos de recuperación** que permitan continuar las operaciones.

Universidad Europea

3. Tolerancia a fallos y alta Disponibilidad

Estrategias Principales

1. Distribución y replicado de datos.
2. Balanceo de carga.
3. Alta disponibilidad y recuperación ante desastres.
4. Sistemas de mensajería y colas de trabajo.
5. Monitorización y gestión de clústeres.

3.1. Distribución y replicado de datos

Distribución

- Los datos se dividen en fragmentos (particiones o shards) que se reparten entre múltiples nodos del clúster.
- **Ventajas**:
  - Acelera el procesamiento al dividir las tareas.
  - Mejora la disponibilidad: si un nodo falla, otros pueden continuar el trabajo.

***Ejemplo**: HDFS → los archivos se dividen en bloques de 128 MB que se distribuyen por los nodos del clúster.*

3.1. Distribución y replicado de datos

Replicación

- Crear copias redundantes de los datos en distintos nodos o incluso en distintas ubicaciones geográficas..
- **Ventajas**:
  - Protección frente a fallos de hardware o pérdida de nodos.
  - Lecturas paralelas desde distintas réplicas.

***Ejemplo**: HDFS → guarda 3 copias de cada bloque de datos en nodos distintos.*

3.2. Balanceo de Carga

- Se reparte de manera ***equitativa el tráfico o las tareas*** de procesamiento entre los distintos nodos del clúster, evitando que unos se sobrecarguen mientras otros permanecen inactivos.
- Esencial para mantener ***rendimiento, estabilidad y eficiencia***.
- Claves del Balanceo
  - *Asignación secuencial* de tareas a cada nodo.
  - Enviar tareas al *nodo con menor carga* actual.
  - *Añadir o quitar nodos* según la carga.

3.2. Balanceo de Carga

- Ventajas
  - Previene cuellos de botella.
  - Mejora el rendimiento global del sistema.
  - Si un nodo cae, las tareas se reasignan automáticamente a otro.

***Ejemplo**: Spark redistribuye tareas automáticamente según la disponibilidad de cada executor.*

3.3. Alta disponibilidad y recuperación ante desastres

Alta Disponibilidad (HA)

- Asegurar que los servicios sigan funcionando sin interrupciones, incluso ante fallos parciales.
- **Claves** de la Alta Disponibilidad:
  - *Redundancia: disponer de varios clústeres.*
  - *Failover automático: redirigir las peticiones a un nodo secundario cuando falla el principal.*
  - *Clusterización activa-activa o activa-pasiva. Todos a la vez o alguno esperando por el fallo.*

3.3. Alta disponibilidad y recuperación ante desastres

Recuperación antes Desastres (DR)

- Garantizar la ***restauración de servicios y datos*** tras fallos catastróficos.
- **Claves** de la Recuperación antes Desastres:
  - *Snapshots y backups*: *copias periódicas de datos almacenadas en distintas ubicaciones.*
  - *Replicación geográfica*: *duplicar datos en diferentes regiones.*
  - *Pruebas de contingencia*: *simulaciones periódicas para verificar la capacidad de recuperación.*

3.4. Sistemas de mensajería y colas de trabajo

- Permiten que distintos componentes del sistema (microservicios, módulos de procesamiento, APIs, etc.) se comuniquen de forma asíncrona y fiable.
- Aportan resiliencia al desacoplar productores y consumidores de datos.
- Claves del Balanceo
  - *Entrega confiable incluso en caso de fallos.*
  - *Replay (releer eventos) y checkpointing.*
  - *Clave en arquitecturas Kappa o Lambda, donde los datos se procesan de forma continua.*

3.4. Sistemas de mensajería y colas de trabajo

Ejemplo Clave: ***Apache Kafka***

- Diseñado para manejar **millones de eventos** por segundo.
- Los datos se envían a **topics** divididos en **particiones**, que se replican entre distintos **brokers**.
- Si un broker falla, otro con una réplica toma el control sin pérdida de datos.
- Kafka ofrece ***persistencia, replicación y confirmaciones*** (acknowledgements) que aseguran la entrega.

3.5. Monitoreo y gestión de clústeres

- La prevención es fundamental. Un sistema bien monitorizado detecta anomalías antes de que provoquen una caída.
- Buenas Prácticas
  - *Definir umbrales de alerta (CPU, memoria, errores, latencia).*
  - *Configurar notificaciones automáticas ante caídas o sobrecargas.*
  - *Aplicar políticas de autoescalado y autorecuperación basadas en las métricas.*

3.6. Comparativa de Estrategias

Estrategia	Objetivo	Ejemplo de tecnología
Replicación	Evitar pérdida de datos	HDFS, Cassandra
Balanceo de carga	Optimizar rendimiento	Zookeeper, HAProxy
Alta disponibilidad	Evitar caídas del sistema	Failover, clústeres redundantes
Mensajería	Coordinación entre nodos	Kafka, RabbitMQ
Monitorización	Supervisar salud del sistema	Grafana, ELK

CONTENIDOS

1. Procesamiento de Datos.
2. Escalabilidad en Big Data.
3. Tolerancia a fallos y alta disponibilidad.
4. **Casos de Uso.**

Universidad Europea

4. Casos de Uso

1. **E-commerce**: análisis de ventas y monitorización de clientes en tiempo real.
2. **Red social**: gestión de millones de publicaciones, fotos y likes por minuto.
3. **Hospital digital**: sensores IoT y monitorización de pacientes 24/7.
4. **Smart city:** datos de tráfico, contaminación y energía.
5. **Plataforma de videojuegos online**: millones de jugadores y eventos simultáneos en tiempo real.
6. **Empresa energética inteligente**: gestión de contadores IoT y predicción de picos de consumo.
7. **Gestión de vuelos**: análisis de telemetría y alertas de seguridad en aeronaves.

Universidad Europea

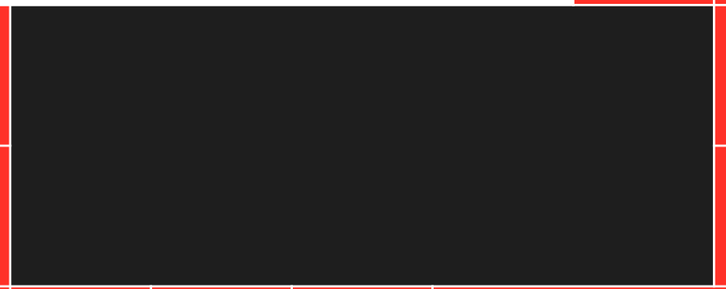
4. Casos de Uso

Preguntas Clave

1. ¿Qué tipo de procesamiento usarías (batch, streaming o híbrido)?
2. ¿Cómo escalamos el sistema?
3. ¿Qué medidas de tolerancia a fallos implementarías y con qué tecnologías?

Universidad Europea

Disfrutemos de lo aprendido



Ve más allá

© Copyright Universidad Europea. Todos los derechos reservados